

Organizing Kubernetes Components with Namespaces

Overview

Namespaces in Kubernetes provide a mechanism for logically isolating and organizing cluster resources.

They help divide cluster resources between:

- Teams
- Applications
- Environments
- Projects
- Departments

Namespaces are especially useful in large Kubernetes clusters where multiple applications and teams share the same infrastructure.

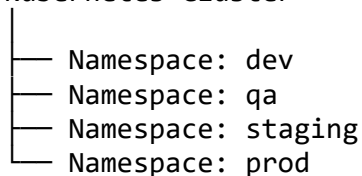
What is a Namespace?

A Namespace is a virtual cluster inside a Kubernetes cluster.

It allows resources to be grouped and managed independently.

Example:

Kubernetes Cluster



Each namespace can contain:

- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- Ingress
- Jobs

- StatefulSets

Why Use Namespaces?

Namespaces provide:

Benefit	Description
Isolation	Separate applications and teams
Organization	Group related resources together
Access Control	Apply RBAC per namespace
Resource Limits	Control CPU and memory usage
Environment Separation	Separate Dev, QA, Prod
Multi-Tenancy	Multiple teams share cluster safely

Default Kubernetes Namespaces

Kubernetes automatically creates several namespaces.

List Existing Namespaces

Run:

```
kubectl get namespaces
```

or

```
kubectl get ns
```

Example output:

NAME	STATUS	AGE
default	Active	20d
kube-system	Active	20d
kube-public	Active	20d
kube-node-lease	Active	20d

Default Namespaces Explained

Namespace	Purpose
default	Default namespace for user workloads
kube-system	Kubernetes system components
kube-public	Publicly accessible cluster information
kube-node-lease	Node heartbeat management

Create a Namespace

Using kubectl Command

Run:

```
kubectl create namespace dev
```

Example output:

```
namespace/dev created
```

Verify Namespace

```
kubectl get namespaces
```

Create Namespace Using YAML

Create a file:

```
namespace.yaml
```

Namespace YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

Apply Namespace YAML

```
kubectl apply -f namespace.yaml
```

Deploy Resources into a Namespace

By default, resources are created in the:

default

namespace.

To deploy resources into a specific namespace, use:

```
kubectl apply -f deployment.yaml -n dev
```

Example Deployment in Namespace

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: dev
```

```
spec:
  replicas: 2
```

```
  selector:
    matchLabels:
      app: nginx
```

```
  template:
    metadata:
      labels:
        app: nginx
```

```
    spec:
      containers:
        - name: nginx
          image: nginx
```

Apply Deployment

```
kubectl apply -f deployment.yaml
```

View Resources in a Namespace

View Pods

```
kubectl get pods -n dev
```

View Deployments

```
kubectl get deployment -n dev
```

View Services

```
kubectl get svc -n dev
```

Set Default Namespace for kubectl Context

Instead of specifying:

```
-n dev
```

for every command, configure the default namespace.

Run:

```
kubectl config set-context --current --namespace=dev
```

Verify Current Namespace

```
kubectl config view --minify | grep namespace:
```

Example output:

```
namespace: dev
```

Switch Back to Default Namespace

```
kubectl config set-context --current --namespace=default
```

Namespace Resource Isolation

Resources inside one namespace are logically separated from other namespaces.

Example:

```
Namespace: dev
├── nginx-service
```

```
Namespace: prod
├── nginx-service
```

Both services can exist with the same name because they are isolated by namespace.

Cross-Namespace Communication

Pods in different namespaces can communicate using:

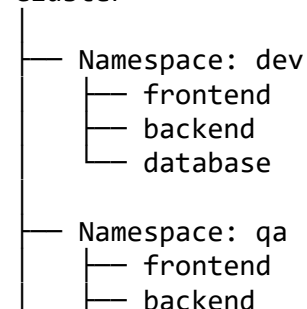
```
<service-name>.<namespace>.svc.cluster.local
```

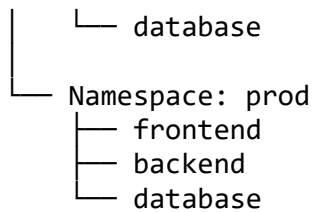
Example:

```
mysql-service.database.svc.cluster.local
```

Example Architecture

Cluster





Delete a Namespace

Run:

```
kubectl delete namespace dev
```

Important:

Deleting a namespace removes all resources inside that namespace.

Resource Quotas in Namespaces

Namespaces support resource quotas.

This prevents teams from consuming excessive resources.

Example ResourceQuota YAML

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev

spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
  
```

Apply ResourceQuota

```
kubectl apply -f resourcequota.yaml
```

View ResourceQuota

```
kubectl get resourcequota -n dev
```

LimitRange in Namespace

LimitRange sets default CPU and memory limits for containers.

Example LimitRange YAML

```
apiVersion: v1
kind: LimitRange
metadata:
  name: resource-limits
  namespace: dev

spec:
  limits:
  - default:
      cpu: 500m
      memory: 512Mi

    defaultRequest:
      cpu: 100m
      memory: 128Mi

  type: Container
```

Apply LimitRange

```
kubectl apply -f limitrange.yaml
```

RBAC with Namespaces

Namespaces allow Role-Based Access Control (RBAC).

Example:

- Dev team access only to dev namespace
 - QA team access only to qa namespace
 - Production access restricted to admins
-

Example Role YAML

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
```

```
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

Best Practices for Namespaces

Use Separate Namespaces for Environments

Recommended:

dev
qa
staging
prod

Separate Teams Using Namespaces

Example:

team-a
team-b
team-c

Apply Resource Quotas

Prevent resource exhaustion.

Use RBAC Per Namespace

Restrict user access appropriately.

Avoid Deploying User Applications in kube-system

System namespace should contain only Kubernetes system components.

Useful Commands

List Namespaces

```
kubectl get ns
```

Create Namespace

```
kubectl create namespace <namespace-name>
```

Delete Namespace

```
kubectl delete namespace <namespace-name>
```

Get Pods in Namespace

```
kubectl get pods -n <namespace-name>
```

Get All Resources in Namespace

```
kubectl get all -n <namespace-name>
```

Switch Default Namespace

```
kubectl config set-context --current --namespace=<namespace-name>
```

Troubleshooting

Resource Not Found

Check namespace:

```
kubectl get pods -A
```

Cannot Access Resource

Check RBAC permissions.

Run:

```
kubectl auth can-i get pods -n dev
```

Namespace Stuck in Terminating State

Check remaining resources:

```
kubectl api-resources --verbs=list --namespaced -o name | xargs -n 1 kubectl  
get -n dev
```

Recommended Namespace Strategy

Namespace	Purpose
dev	Development workloads
qa	QA testing
staging	Pre-production validation
prod	Production applications
monitoring	Prometheus, Grafana
ingress	Ingress controllers
logging	ELK, Fluentd

Summary

Kubernetes Namespaces:

- Organize cluster resources logically
- Provide isolation between applications and teams
- Support RBAC and resource quotas
- Enable multi-tenant Kubernetes clusters
- Improve cluster management and scalability